Viva Cheat Sheet — StMangSys

Facts: Java 25 · Java Swing (NetBeans GUI Builder) · package `stmangsys` · main class `stmangsys.StMangSys` · in-memory object arrays (data resets on exit) · 26 `.java` + 19 `.form` files · Ant project.

Architecture (say this first)

View (Swing forms: validate input, JOptionPane, fill JTables) → **Access layer** (Singletons: one shared data source for every form) → **Model/Logic** (`Student`, `StudentCollection`, `Batch`, `BatchCollection` — all rules and algorithms, zero Swing code). **Why Singleton:** every form shares ONE copy of the data → a student added in one window shows instantly in every table/report. Private constructor + `static getInstance()` with lazy creation.

Key rules & formats

- **Registration no** = `PR|OR` + `24` + batchNo + 3-digit sequence, e.g. `PR24101004`. System-generated; sequence is **batch-wide** (highest in batch + 1), PR/OR = Physical/Online lecture mode.
- **Batch numbers** auto-generate from 100 upward; add allowed only to **OPEN** batches (combo pre-selects first open batch; closed batch → blocked with message).
- **Student update** = name + NIC only (batch fixed by spec). **NIC must be unique** (duplicate shows existing profile).
- **Marks:** `−1` = ABSENT (needs confirmation), `−2` = NOT CONDUCTED (default on add), otherwise must be 0–100 (150 → rejected).
- **Student GPA = (PRF GPA + DBMS GPA) ÷ 2**; a subject without a real mark contributes **0**.
- **Training eligibility:** GPA **> 3.25** AND **both** PRF > 50 AND DBMS > 50.

GPA value table: 90–100 → **4.25** · 80–89 → **4.00** · 75–79 → **3.70** · 70–74 → **3.30** · 65–69 → **3.00** · 60–64 → **2.70** · 55–59 → **2.30** · 50–54 → **2.00** · 45–49 → **1.70** · 40–44 → **1.30** · 30–39 → **1.00** · 0–29 → **0.70** · absent/not-conducted → **0**. (if/else-if ladder, highest band first)

Worked numbers (whiteboard-ready)

PRF	DBMS	PRF GPA	DBMS GPA	Student GPA	Eli
94	39	4.25	1.00	**2.625** (spec sample)	No (DI ≤ 5
85	80	4.00	4.00	**4.0**	**Ye**
70	70	3.30	3.30	**3.3**	**Ye** > 3
65	65	3.00	3.00	**3.0**	No ≤ 3
–2	–2	0	0	**0**	No

Reports (spec section 4)

All tables show: **Batch No, Reg No, PRF Marks, DBMS Marks, GPA** (marks as `ABSENT` / `NOT CONDUCTED`), sorted **alphabetically by name**, each with **Back to Home Page**.

1. **Student Registration Report** — all students. 2. **Batch-wise** — selection page with one button per batch ("101 Batch", action command = batch no) → "101 Batch Student Report". 3. **Industry Training Eligibility** — filter by the criteria above. Reports reload from the singleton **on construct**; the hub disposes itself when opening a report, so "Back to Home Page" reveals the home page.

Swing & NetBeans theory

- Event-driven: clicks fire `ActionEvent` → registered `ActionListener.actionPerformed` — all on the **EDT**; GUI started via `EventQueue.invokeLater`.
- `.form` XML = GUI Builder design file; regenerates `initComponents()` inside `//GEN` markers (never edit); layouts are **GroupLayout** (Builder) — `SelectBatchForm` switches a panel to **BoxLayout PAGE_AXIS** at runtime for vertical batch buttons.
- `pack()` sizes to preferred sizes; `setLocationRelativeTo(null)` centers; `setResizable(false)`.
- **DISPOSE_ON_CLOSE** = release one window (all child forms) vs **EXIT_ON_CLOSE** = end JVM (main form; Close button also `System.exit(0)`).
- `JTable` + **`DefaultTableModel`**: `setRowCount(0)` then `addRow(...)`; wrapped in `JScrollPane`.
- `JOptionPane`: `showMessageDialog` (warnings/success), `showConfirmDialog` + YES_NO (delete, mark ABSENT).

OOP & algorithms checklist

Encapsulation (private fields + getters/setters) · **Singleton** · composition (collection has-a array of students) · class constants

(`MARK_ABSENT=-1`, `MARK_NOT_CONDUCTED=-2`, `STATUS_OPEN=1/CLOSED=0`) · parameterized constructor (`BatchWiseStudentReport(batchNo)`) · `toString()` override. Arrays: grow by copying to a bigger array · **linear search** O(n) with `equalsIgnoreCase` · delete by shifting into a smaller array · **defensive copies** (`getAllStudent` returns a copy) · **insertion sort** by name (`compareToIgnoreCase`).

Add-student validation chain (in order)

batch selected → batch exists → enrollment **OPEN** → NIC not empty → name not empty → **NIC unique** (else show existing student) → generate regNo → add → success dialog with reg no.

Code you must be able to explain

```
// Singleton
public static
        AccessStudentCollection
        getInstance() {
    if (accessStudentCollection
        == null)
        accessStudentCollection =
        new
        AccessStudentCollection();
    return
        accessStudentCollection;
}
// GPA = (PRF GPA + DBMS GPA) /
        2, absent counts 0
double prfGpa  =
        Math.max(marksGpa(prfMarks),
        0);
double dbmsGpa =
        Math.max(marksGpa(dbmsMarks),
        0);
return (prfGpa + dbmsGpa) /
        2.0;
// Registration no: PR|OR + 24
        + batchNo + zero-padded
        highest-seq+1
// Insertion sort for report
        order:
while (j >= 0 &&
        students[j].getName().compareToIgnoreCase(key.getName())
        > 0)
    students[j + 1] =
        students[j];
// Eligibility:
return getGpa() > 3.25 &&
        isRealMark(prfMarks) &&
        prfMarks > 50
                    &&
        isRealMark(dbmsMarks)
        && dbmsMarks > 50;
```

Rapid-fire Q&A

- **Where is data stored?** In-memory arrays in the singletons; resets on exit (file/DB = future work).
- **Which thread runs button handlers?** The Event Dispatch Thread (EDT).
- **Why arrays, not ArrayList?** Coursework style; demonstrates manual growth, copying, shifting.
- **What is `initComponents()` / `.form`?** Builder-generated setup; `.form` is its design XML.
- **Why batch-wide sequence?** Spec IDs run 001,002,003… per batch regardless of PR/OR.
- **Absent subject in GPA?** Counts as 0 — reproduces spec sample (94 + 39 → 2.625).
- **Why insertion sort?** Small data, simple, stable, plain-array coursework style.
- **How was it tested?** Automated UI driver walked every screen: **32/32 checks** (validations, GPA 2.5 & 2.625, ID `PR24101004`, closed-batch block, report contents) + 46 screenshots.

Demo order & run

Home → Student Mgmt: add student (**PR24101004** appears in View-All table) → Grade Mgmt: 94/39 → profile **2.625** → Batch Mgmt: add 103, close it → Report Generator: 3 reports → Close. **Run:** NetBeans F6, or `javac -d build/classes $(find src -name "*.java")` then `java -cp build/classes stmangsys.StMangSys`.